

FontFusion

Project Team

Alishba Nadeem 21I-0879
Hareem Sherjeel 21I-0783
Amna Aiman 21I-2743

Session 2021-2025

Supervised by

Dr. Hasan Mujtaba

Co-Supervised by

Ms. Zonera Anjum



Department of Computer Science

**National University of Computer and Emerging Sciences
Islamabad, Pakistan**

October, 2024

Contents

1	Introduction	1
1.1	Problem Domain	1
1.2	Research Problem Statement	1
1.2.1	Software Requirements Specification	2
1.2.1.1	Functional Requirements	2
1.2.1.2	Non-Functional Requirements	2
2	Literature Review	3
2.1	Related Research Papers	3
2.1.1	Application and Difference of Raster and Vector Graphics [3] . . .	3
2.1.1.1	Summary	3
2.1.1.2	Critical Analysis	4
2.1.1.3	Relationship to the proposed research work	4
2.1.2	TrueType Transformer: Character and Font Style Recognition in Outline Format [4]	4
2.1.2.1	Summary	4
2.1.2.2	Critical Analysis	5
2.1.2.3	Relationship to the proposed research work	5
2.1.3	Large-Scale Font Identification from Document Images [1]	5
2.1.3.1	Summary	5
2.1.3.2	Critical Analysis	6
2.1.3.3	Relationship to the proposed research work	6
2.1.4	DeepFont: Identify Your Font from An Image [5]	6
2.1.4.1	Summary	7
2.1.4.2	Critical Analysis	7
2.1.4.3	Relationship to the proposed research work	7
2.1.5	Raster-to-Vector: Revisiting Floorplan Transformation [2]	7
2.1.5.1	Summary	8
2.1.5.2	Critical Analysis	8
2.1.5.3	Relationship to the proposed research work	8
2.2	Analysis Summary of Research Papers	9
2.3	Existing Font Conversion Apps	10

2.4	Feature Comparison	11
3	Proposed Approach	13
3.1	Font Feature Extraction and Preprocessing	13
3.2	Classification of Fonts	14
3.2.1	Model Choice	15
3.2.2	Training Process	15
3.3	Font Prediction Model	15
3.3.1	Similarity Metric	15
3.3.2	Font Matching Algorithm	16
3.4	Website Development	16
3.4.1	User Interface and Workflow	16
3.4.2	Backend System and Technology	17
3.4.3	Advantages and Benefits	17
3.4.4	Tentative Visual Representation of the UI Interface	17
3.5	Graphical Representation of the Approach	20
	References	23

List of Figures

3.1	Home Page - 1	18
3.2	Home Page - 2	18
3.3	Font Comparison Results Page	19
3.4	About Us Page	19
3.5	Contact Us Page	20
3.6	Methodology Diagram	20

List of Tables

2.1	Analysis of Related Research Papers	9
2.2	Existing Font Conversion Apps with Strengths and Weaknesses	10
2.3	Feature Comparison with Our App - FontFusion	11

Chapter 1

Introduction

The company in collaboration with our project utilize old-fashioned raster fonts on their machines. These fonts were effective, in times. But they are now inadequate for modern high-resolution screens. This problem has emerged as an obstacle, for the company operations and it undermines the overall user experience quality.

Raster fonts lose quality when increased in size because they are composed of fixed pixel grids. They are designed at a specific resolution and lack scalability, resulting in a blurry or pixelated appearance when their size is increased e.g. from 12pt to 14pt. Unlike vector fonts, which use mathematical definitions to maintain clarity at any size, raster fonts are resolution-dependent and cannot render smoothly beyond their original pixel data, leading in low visual quality and reduced sharpness on high-resolution displays.

This report outlines the problem, motivation to solve that problem and a proposed solution which guarantees that the company's technology remains up to date and in line, with current digital norms.

1.1 Problem Domain

Raster fonts lack flexibility due to their fixed pixelated size and has tendency to lose sharpness when the size is increased. Vector fonts, like OTF (Open-Type Font) and TTF (True-Type Font) do not have that problem. Our problem domain is to focus on this limitation of raster fonts in today's high resolution screens.

1.2 Research Problem Statement

The core issue that this research covers is the limitation of raster fonts in high-resolution screens because their pixelated structure does not scale well when resized.

The project's goal is to compare the raster fonts with these vector fonts, classify them in terms of size, shape and style and predict the font outline. By automating the task of font classification, this solution will facilitate the company's shift, to similar font adjustments that maintain clarity and uniformity, across different screen dimensions.

With our proposed solution implemented, the raster fonts visual and resolution issue will be successfully solved and they can be converted to similar scalable vector fonts found by our designed app that look and feel like their existing font style in the future.

1.2.1 Software Requirements Specification

The software developed in this project must meet the following requirements:

1.2.1.1 Functional Requirements

- Automate the identification and extraction of features from raster fonts by means of neural networks.
- Comparison of raster fonts' extracted features with a library of TTF/OTF fonts to find the closest match in terms of shape, size and style.
- A user-friendly web interface should be provided so that raster font images can be uploaded and the similar vector font can be shown in results.

1.2.1.2 Non-Functional Requirements

- The number of fonts should be handled efficiently by the software. It should be able to give real-time results.
- The system should offer an easy interface, without any technical requirements from the user.
- The system should have the ability to upgrade to more fonts inputs and comparisons if needed.

Chapter 2

Literature Review

This chapter critically examines existing literature relevant to our research topic. It identifies key studies, theories, and findings, providing a foundation for the proposed research and highlighting its relationship to the prior work.

2.1 Related Research Papers

2.1.1 Application and Difference of Raster and Vector Graphics [3]

This research paper focuses on the characteristics, applications, and distinctions between raster and vector graphics, offering insights into how these two types of computer graphics/fonts are used across various domains.

2.1.1.1 Summary

The paper presents an in-depth overview of raster and vector graphics, emphasizing their fundamental differences. Raster graphics are described as pixel-based images, where each pixel has its own defined color, forming the image in a grid-like structure. Vector graphics, on the other hand, utilize geometric shapes and mathematical equations to create images. This makes them scalable without loss of quality, a significant advantage over raster images. The study also discusses image formats, such as BMP, JPEG for raster and EPS, AI, and SVG for vector graphics. It elaborates on the software tools commonly used for each type, with Adobe Photoshop and Corel Photo-Paint being key for raster images, while Adobe Illustrator, Corel Draw, and CAD programs are integral to vector graphics.

2.1.1.2 Critical Analysis

The strength of this paper lies in its clear differentiation between raster and vector graphics, which is essential for users needing to decide which type of image to use in specific applications. The comparison of image formats and software tools adds practical value. However, the paper tends to focus more on the technical aspects without deeply exploring the evolving trends in hybrid graphics technologies or how advances in compression algorithms and hardware have started to blur the boundaries between raster and vector graphics. Furthermore, while the paper offers detailed descriptions of graphic types, it lacks practical data or case studies that could provide more concrete examples of their applications in modern industries.

2.1.1.3 Relationship to the proposed research work

The topic of raster and vector graphics in this study is directly related to the proposed research on feature extraction in raster font images using TTF files. The exploration of how raster images handle pixel information and the comparison of resolution scalability offers valuable background information for understanding the challenges in feature extraction of raster fonts from their images. Additionally, the software tools mentioned in the paper, such as Adobe Photoshop and Corel Photo-Paint, align with the research focusing on the processing of raster images. Information about the vector images and the technical comparisons will help in understanding image processing within the study.

2.1.2 TrueType Transformer: Character and Font Style Recognition in Outline Format [4]

This paper reviews the TrueType Transformer (T3), a model designed to classify fonts and recognize character and style based on outline formats.

2.1.2.1 Summary

The study proposes a novel approach to font recognition that uses outline-based data rather than pixel-based images/data. The model is intended to process control points that determine the outlines of characters, allowing recognition of both character and font styles. By visualizing attention weights in the Transformer model, the study determines which control points are most significant for classification, providing greater insights into font/typeface structure. The study also compares T3's performance to traditional approaches such as ResNet, finding comparable accuracy for character identification and an advantage in style recognition due to precise structural information from control points. It

concludes with possible applications for more delicate classification tasks, such as identifying font impressions.

2.1.2.2 Critical Analysis

The TrueType Transformer model has the strengths of Transformer architectures in handling variable-length sequences, which are well-suited to font outlines represented by control points. One of the key strengths of the study is the ability to visualize attention weights, providing transparency into how the model interprets font styles. Additionally, T3’s adaptability to different font styles demonstrates its versatility. However, one major shortcoming is the underrepresentation of off-curve points, which are not discussed properly, despite of their potential importance in more sophisticated classifications. The study’s focus on only four broad font styles limit the generalizability of its findings to more complex or subtle font distinctions.

2.1.2.3 Relationship to the proposed research work

The T3 model’s emphasis on control points for font classification can significantly enhance the proposed research by helping in the extraction of structural features from raster fonts during CNN training. This directly addresses the core issue of raster fonts’ limitations on high-resolution screens, where pixelation limits scalability. By automating font classification, the research aims to provide alternate scalable vector fonts that maintain clarity across various dimensions. Additionally, exploring the handling of off-curve points in the T3 model may provide an opportunity to improve recognition tasks based on subtle structural cues.

2.1.3 Large-Scale Font Identification from Document Images [1]

This paper addresses the challenge of font identification in document images and emphasizes the complexities of visual font recognition, and proposes data augmentation techniques such as scaling and rotation to enhance model robustness.

2.1.3.1 Summary

The research presents a systematic approach to font identification through the use of a CNN model trained on a comprehensive dataset of 10,000 fonts. The authors first preprocess document images to extract individual characters and then apply a max-voting policy to improve recognition accuracy. The experiments conducted on synthetic and natural

document images demonstrate the model’s effectiveness in distinguishing between visually similar fonts. The paper concludes that the proposed method offers a robust solution to the visual font recognition problem, particularly for applications in design editing and automated text translation.

2.1.3.2 Critical Analysis

One of the strengths of this study is its comprehensive dataset, which allows for an extensive evaluation of the font recognition problem across a diverse range of font types. The CNN architecture employed demonstrates promising results, showcasing the effectiveness of deep learning techniques in handling the complexities of font identification. However, the study also has notable weaknesses. While the model performs well on synthetic images, its performance may vary in real-world scenarios due to the inherent challenges of document image degradation, noise and variations in font styles. Additionally, the paper does not thoroughly address the limitations of its methodology in distinguishing between closely related font styles, which is a critical aspect in practical applications.

2.1.3.3 Relationship to the proposed research work

The findings of this paper resonate significantly with our research on raster font classification. Both studies tackle the complexities of font identification, particularly in distinguishing between visually similar font types. The CNN-based approach highlighted in this paper aligns with our intention to utilize deep learning techniques for classifying raster fonts based on their characteristics and classify them with respect to their similarity to vector fonts. Moreover, the paper’s emphasis on data augmentation methods, such as scaling and rotation, is particularly relevant to our work, as these techniques can enhance the robustness of our model against variations in raster font images. By identifying and addressing the challenges related to font style distinctions mentioned in the study, we can refine our methodology for classifying raster fonts, ultimately contributing to the development of an effective solution that facilitates the transition to scalable vector fonts without compromising visual integrity.

2.1.4 DeepFont: Identify Your Font from An Image [5]

This section reviews the paper by summarizing its contributions and evaluating its relevance to the proposed study on font recognition and classification.

2.1.4.1 Summary

The paper presents a deep learning framework designed to identify fonts in images. The authors developed a system that uses convolutional neural networks (CNNs) to extract font features from images of text. The methodology includes training a deep network using a large dataset of labeled fonts, followed by performance evaluation on various test images. The main findings indicate that the proposed method achieves high accuracy in font recognition tasks, outperforming existing techniques. The study concludes that the DeepFont system can effectively identify fonts in images, thus providing a significant advancement in the field of font recognition.

2.1.4.2 Critical Analysis

The strengths of this research include its innovative approach to using deep learning for font identification, which demonstrates the potential of CNNs in processing visual data. Adobe TypeKit dataset is used for CNN Training, which contains a diverse collection of font styles and variations. The dataset includes more than 1,000 fonts, capturing a wide range of typographic styles. However, the study has certain weaknesses, such as limited exploration of real-world scenarios where images may contain noise or distortions, which can affect accuracy. Additionally, relying on labeled datasets may introduce biases, as the model's performance is dependent on the quality and diversity of the training data.

2.1.4.3 Relationship to the proposed research work

The findings from the DeepFont study significantly inform the proposed research on font classification. The use of deep learning techniques parallels the objectives of this study, particularly in automating font classification and identifying distinct features among raster and vector fonts. Furthermore, the gaps identified in the DeepFont paper, such as handling noisy images and dataset bias, highlight areas for improvement in the current research. By building upon the methodologies of DeepFont while addressing its limitations, this study aims to enhance raster font classification systems, thus contributing to the modern font technologies.

2.1.5 Raster-to-Vector: Revisiting Floorplan Transformation [2]

This is a github repository which holds a great importance in our study. As it provides a model for raster to vector transformation.

2.1.5.1 Summary

The paper presents an effective algorithm for converting rasterized floorplan images into vector-graphics representations. The proposed method achieves approximately 90 percent precision and recall, positioning it as a production-ready solution. The algorithm is implemented in Torch7, with updates provided for a PyTorch version, which simplifies the setup process. The authors also offer a free integer programming solver and discuss a follow-up project focused on reconstructing floorplans from 3D scans. Due to data-sharing restrictions, only vector-graphics annotations from the LIFULL dataset are made available, which can support various tasks.

2.1.5.2 Critical Analysis

This research exhibits several strengths, including its high performance metrics (90 percent precision and recall) and the clear requirements and usage instructions. The provision of both Torch and PyTorch implementations facilitates broader accessibility. However, the limitations arise from the inability to share the original raster images, which may restrict the applicability of the model and code provided. Additionally, while the follow-up project demonstrates the versatility of the authors' approach, it raises questions about the generalizability of the algorithm across different types of floorplan images and environments. Furthermore, the reliance on specific hardware (Nvidia Titan GPU) may limit accessibility for researchers with different setups.

2.1.5.3 Relationship to the proposed research work

The findings from this research are highly relevant to the proposed study on raster font recognition and classification, particularly regarding the challenges of converting visual data (raster images) into structured formats (vector graphics). The methodology outlined in this paper can help in the design of similar algorithms specifically for raster fonts, emphasizing the importance of high precision and recall in achieving reliable performance. Additionally, the use of deep learning techniques, particularly convolutional neural networks (CNNs), aligns with the approaches being considered for automating font classification.

2.2 Analysis Summary of Research Papers

The table below captures key points of each research paper. The strengths and weaknesses are evaluated to determine how each study influences the methodology of the current research.

Research Item	Strengths	Weaknesses	Relevance to Current Research
Application and Difference of Raster and Vector Graphics [3]	Clear differentiation between raster and vector graphics; practical value in image formats and software tools.	Focus on only technical aspects; lacks practical data/case studies; limited exploration of hybrid graphics technologies.	Directly relates to feature extraction challenges in raster fonts; contributes to raster images processing techniques.
TrueType Transformer: Character and Font Style Recognition in Outline Format [4]	Utilizes control points for font classification; visualizes attention weights; adaptable to different font styles.	Underrepresentation of off-curve points; limited to only four broad font styles.	Helps in building foundation for structural feature extraction from raster fonts; addresses raster fonts' pixelation issues.
Large-Scale Font Identification from Document Images [1]	Comprehensive dataset for extensive evaluation; effective CNN architecture for font identification.	Performance may vary with real-world document image challenges; limitations in distinguishing closely related font styles.	Resonates with raster font classification; aligns with deep learning techniques to enhance robustness against font variations.
DeepFont: Identify Your Font from An Image [5]	Innovative deep learning approach; high accuracy in font recognition; diverse dataset.	Limited exploration of real-world scenarios; potential biases from labeled datasets.	Provides insights into automating font classification; highlights areas for improvement in handling noisy images.
Raster-to-Vector: Revisiting Floorplan Transformation [2]	High precision and recall; clear usage instructions; dual implementations in Torch and PyTorch.	Inability to share original raster images; potential limitations in generalizability across different environments.	Relevant for designing algorithms for raster font classification; emphasizes precision in performance metrics.

Table 2.1: Analysis of Related Research Papers

2.3 Existing Font Conversion Apps

The table below shows the strengths and weaknesses of the existing font conversion apps related to our research.

App(s)	Strengths	Weaknesses
Snowb Bitmap Font	Allows users to upload a font file and generate an image of custom text rendered in the uploaded font.	The image is compressed to a fixed size, and the background has a chess-like texture.
Autotracer.org	Converts raster images like JPEGs, GIFs, and PNGs to scalable vector graphics (EPS, SVG, AI, and PDF).	Tested with a raster image, no accuracy.
Font Converter	Converts many font formats to the most popular formats like TTF, OTF, and WOFF.	Primarily for format conversion, not focused on providing font styling or visualization tools, does not convert raster font to other fonts.
FontDrop!	Takes TTF files as input and provides great tools for visualizing (waterfall/text) and testing custom fonts. Allows detailed glyph analysis.	Does not provide suitable images for the rendered font text.

Table 2.2: Existing Font Conversion Apps with Strengths and Weaknesses

2.4 Feature Comparison

Feature	Snowb Bitmap Font	Auto tracer .org	Font Con- verter	Font Drop!	Font Fusion*
Detect Text Image in Raster Font	✗	✗	✗	✗	✓
Raster and Vector font Comparison	✗	✗	✗	✗	✓
Good Accuracy related to Raster Font	✓	✗	✗	✗	✓
Good Quality image rendering	✗	✗	✗	✓	✓
User-Friendly Aesthetic Website/App	✗	✗	✗	✓	✓

Table 2.3: Feature Comparison with Our App - FontFusion

*our app

Chapter 3

Proposed Approach

A way forward is to find modern vector fonts (TrueType Formal/Open Type Format) that resemble with the existing raster fonts currently in use. Our assigned project is to research and develop a web-based solution that facilitate the client and automatically search for the latest font to which the old undesired Raster fonts are similar. The process will be to compare them to a library of TTF/OTF fonts, and find the best match in terms of their font properties like shape, size and style.

The methodology that we have determined till now from relevant research papers includes the following steps:

3.1 Font Feature Extraction and Preprocessing

The first step of the approach involves extracting key visual features from both raster and vector fonts. The challenge lies in analyzing raster font images, which are pixel-based, and comparing them with vector fonts like TrueType Fonts (TTF) and OpenType Fonts (OTF), which are defined by mathematical outlines. The process includes:

- **Data Generation:**

The generation of font data is an essential step in building a diverse dataset for model training and testing. In this project, existing raster fonts are generated programmatically using Python scripts, which render characters from raster font files (e.g., TTF or OTF formats) into images containing text in raster font. The process involves:

- **Raster Font Generation from Font Files:** A Python script utilizing libraries such as *Pillow* is used to create raster format images at different resolutions. The script loads raster font file (TTF/OTF), and each character is rendered as

a raster image by specifying the desired pixel size and resolution. By generating raster fonts at varying sizes, the system captures the visual distortion and pixelation that occurs during scaling, which is critical for understanding raster font limitations.

- **Multiple Resolutions and Styles:** To enhance the dataset diversity, the fonts are rendered at multiple resolutions (e.g., low to high pixel densities) and in various styles such as bold, italic, or underlined. These variations allow the system to learn how different styling and scaling affect raster font clarity and sharpness.
- **Data Augmentation:** Further augmentations such as rotation, translation, and noise addition are applied to the raster images to simulate real-world distortions. These transformations ensure that the model can generalize well and is robust to minor variations in the input images.
- **Labeling:** Each generated raster font image is labeled with metadata such as the font family, size, style (e.g., bold, italic), and resolution. This labeling is critical for later stages of classification, where the model will need to associate visual features with specific font characteristics.

The vector font dataset will be taken from Google Fonts online.

- **Image Segmentation:** For raster fonts, the character images need to be preprocessed to isolate individual letters and remove noise. This may involve converting the images to grayscale, binarization, and edge detection to extract the character outlines. It's important to note that the specific preprocessing steps may vary depending on the characteristics of the input images.
- **Vector Data Parsing:** For vector fonts, the font files (in .ttf or .otf formats) will be parsed to retrieve their control points and outlines, which define the curves and shapes of the characters.
- **Feature Selection:** Features such as aspect ratio, stroke width, curve smoothness, and corner sharpness are extracted from both raster and vector fonts. These features provide a basis for comparison and classification, as they define the visual style of each font and are critical for the classification process.

3.2 Classification of Fonts

After extracting relevant features, the next step is to classify the fonts based on their style, shape, and size. A supervised machine learning algorithm will be employed to learn the characteristics of different fonts and classify them accordingly.

3.2.1 Model Choice

The classification model is crucial for recognizing patterns in both raster and vector font features. We will consider using a *Support Vector Machine (SVM)* or a *Convolutional Neural Network (CNN)*, noting that SVM may be preferable for simpler, linearly separable data, while CNNs excel in handling high-dimensional, image-based data.

3.2.2 Training Process

The classifier will be trained on font datasets, where each font is labeled with its type (raster or vector), size, style, and shape. The training process involves:

- **Data Augmentation:** For raster fonts, variations in scale and rotation can be applied to improve the robustness of the model.
- **Cross-Validation:** K-fold cross-validation will be used to ensure the model generalizes well across different font styles and sizes.
- **Accuracy Metrics:** The model's performance will be evaluated based on classification accuracy, precision, recall, and F1 score to ensure accurate font classification.

3.3 Font Prediction Model

Once the fonts have been classified, the next task is to predict an equivalent vector font for a given raster font. The prediction model will take the extracted features of a raster font as input and output the most similar vector font in terms of size, shape, and style. This process will involve:

3.3.1 Similarity Metric

A similarity function will be defined to compare the feature vectors of the raster font and the vector fonts. The *Euclidean distance* between the feature vectors will serve as a measure of how closely the raster font resembles a particular vector font. It is important to specify that "size" will be assessed based on dimensions and pixel density for accurate comparison.

3.3.2 Font Matching Algorithm

Based on the similarity metric, a heuristic-based font matching algorithm will be developed to recommend a vector font that matches the raster font. The system will map raster font features to their corresponding vector features and predict a vector font that maintains the closest visual consistency.

3.4 Website Development

To complement the font classification and prediction model, a simple yet effective web application will be developed. This tool will allow users to upload raster font images and receive high-quality, closely matched vector font results. The application aims to make the font conversion process easy and accessible, saving users significant time and effort, especially in scenarios where maintaining visual consistency and clarity across different resolutions is crucial.

3.4.1 User Interface and Workflow

The web application will have a user-friendly interface, designed for both technical and non-technical users. The workflow will be as follows:

- **Font Image Upload:** Users can upload images containing raster fonts. The application will support multiple image formats such as PNG and JPEG. Each image will undergo preprocessing to isolate individual characters and remove noise if necessary.
- **Font Matching and Classification:** Once the raster font image is uploaded, the backend system will process the image using the trained classification model to identify the general type of font, while the prediction model performs the actual matching. The system will extract relevant features from the raster font image and compare them with the features of vector fonts in the database.
- **Prediction Results:** The system will output the predicted vector font (or fonts) that best matches the uploaded raster font in terms of features extracted from the input image such as style, size, and shape. The user will receive a new page for the vector font display results.
- **Additional Font Information:** Along with the predicted font, the web application will also provide additional information about the matched font, including its family, style (e.g., serif, sans-serif), and characteristics (e.g., weight, stroke width).

This metadata will help users understand the matching process and refine their font selection if needed.

3.4.2 Backend System and Technology

The backend system will handle all the processing, including image analysis, feature extraction, font classification, and outline prediction. The system will be built using modern web development frameworks such as Flask (for Python integration) and will be connected to a machine learning model deployed on a server. The font classification and prediction model will be trained offline and integrated into the web application for real-time font matching.

3.4.3 Advantages and Benefits

This web application offers several advantages:

- **Time-Saving:** Users can quickly upload raster font images and get precise vector font suggestions without manual searching or testing.
- **Accessibility:** With the simple interface, the application is accessible to designers, developers, and non-technical users alike, making font comparison straightforward.
- **Consistency and Clarity:** The application ensures that raster fonts are compared with scalable vector fonts that match the original font image's clarity, improving visual quality across different screen resolutions.
- **Scalability:** The backend system is designed to handle a high volume of font image uploads and predictions, making it scalable for multiple users at once.

3.4.4 Tentative Visual Representation of the UI Interface

The following images depict the proposed user interface for the FontFusion website, designed to facilitate the seamless integration of raster and vector fonts. Each page is crafted with user experience in mind, ensuring easy navigation and accessibility to key functionalities. This visual representation illustrates our commitment to creating a user-friendly platform that caters to the needs of our users.

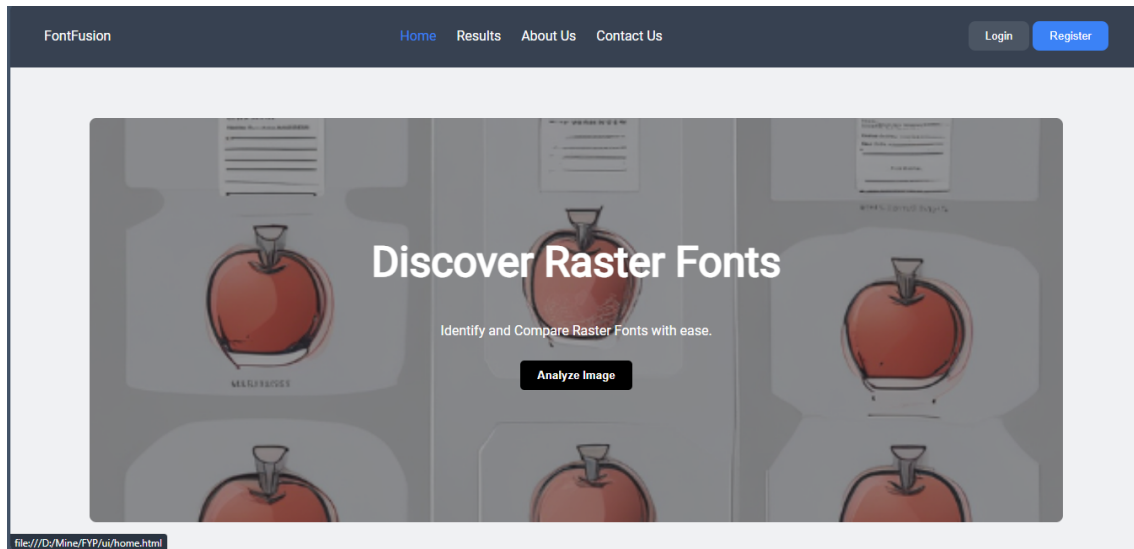


Figure 3.1: Home Page - 1

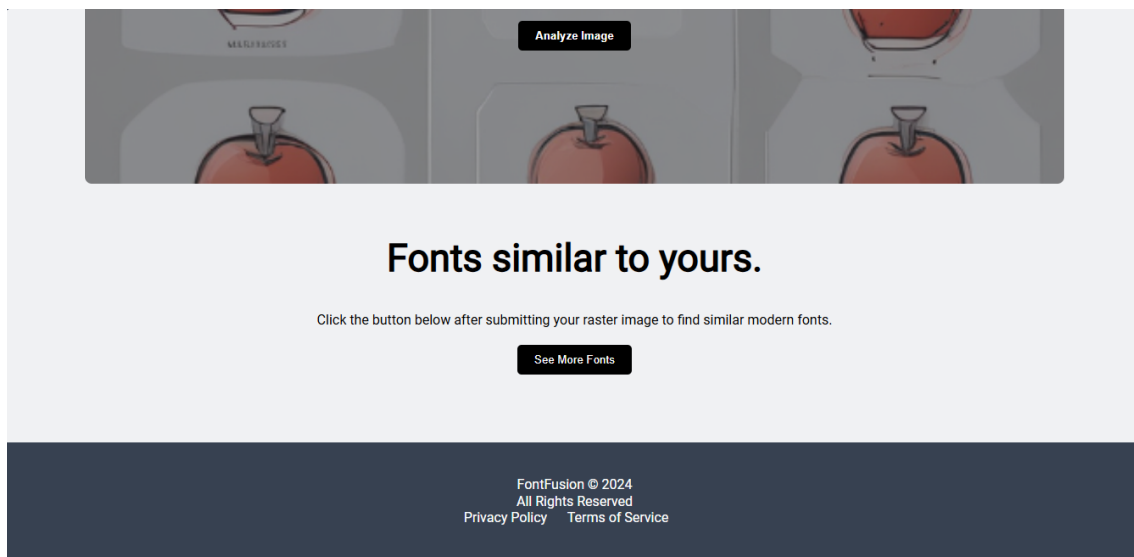


Figure 3.2: Home Page - 2

The Home Page will allow users to input/submit a file of their image which will contain text in Raster Font. Users can also navigate to other sections of the website from here.

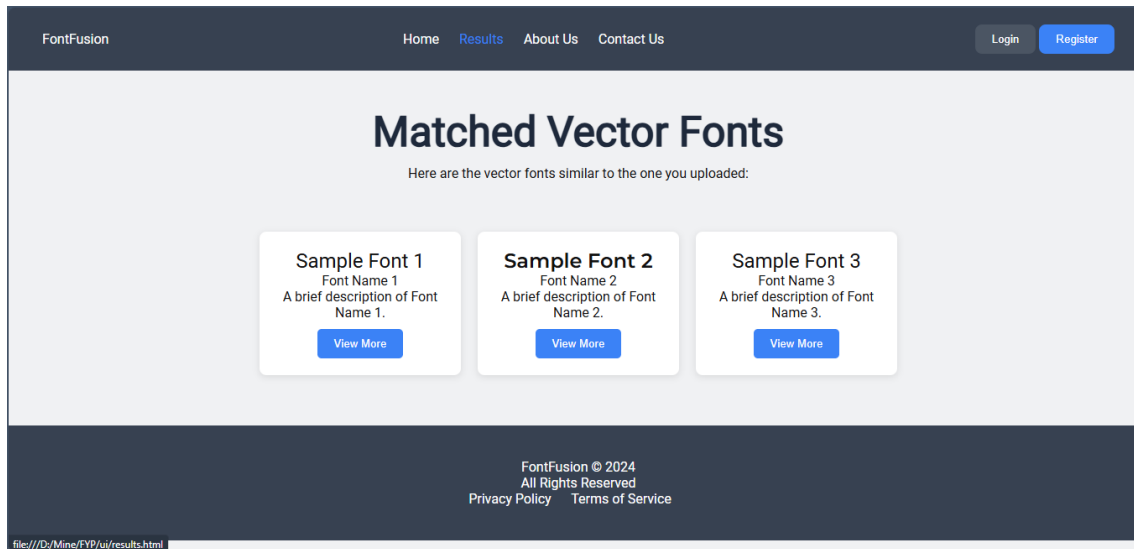


Figure 3.3: Font Comparison Results Page
elfig:resultspage

The Results Page will show the image font comparison results with vector font libraries. This feature allows users to check which fonts are similar to their uploaded Raster font text image.

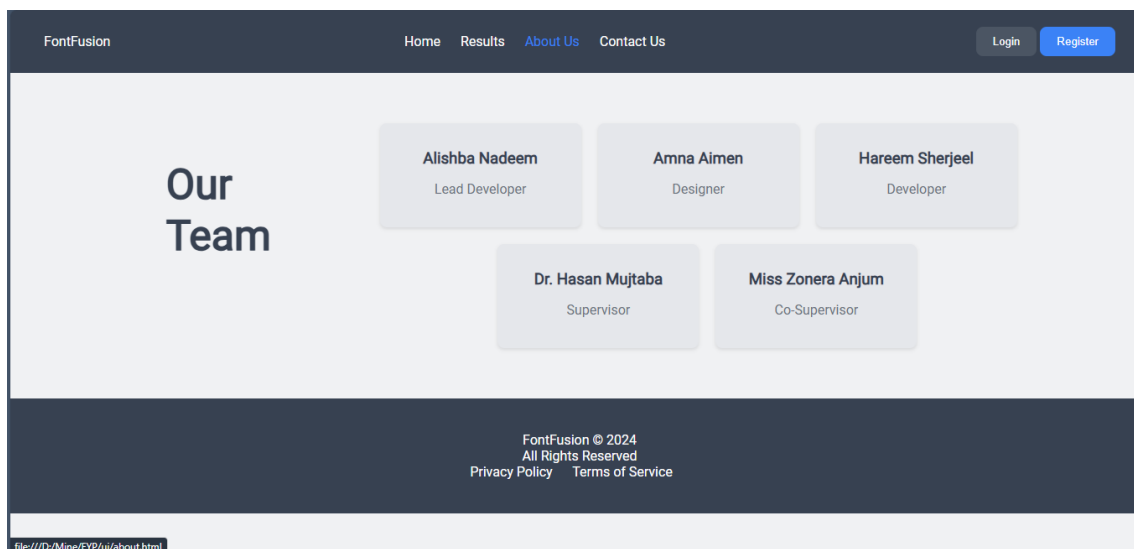


Figure 3.4: About Us Page

The About Us Page shows information about the FontFusion project and the team. Users can learn about the mission, vision, and development process.

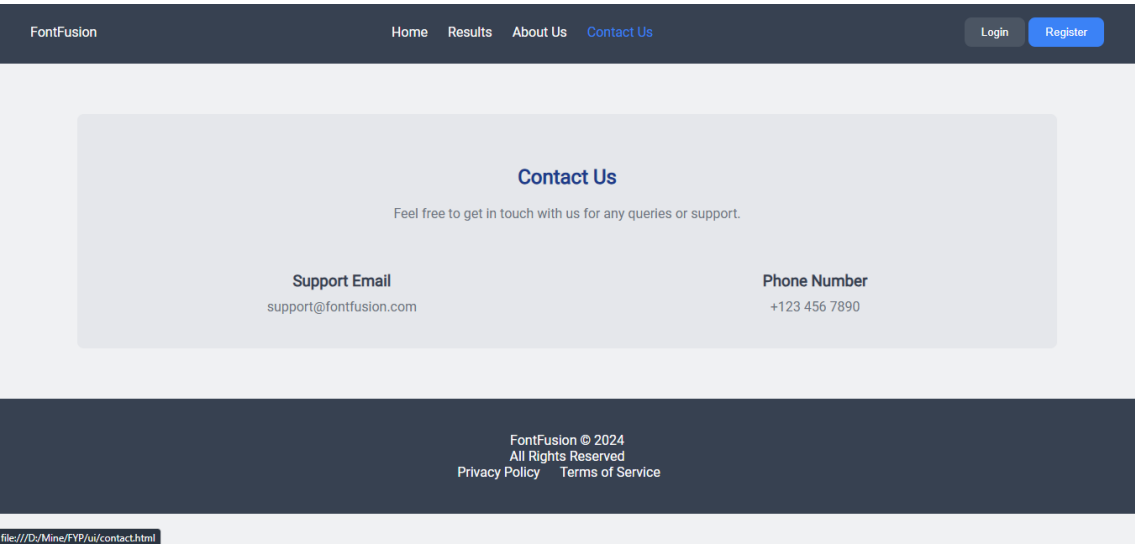


Figure 3.5: Contact Us Page

The Contact Us Page enables users to reach out to the team for support or inquiries.

3.5 Graphical Representation of the Approach

Figure 3.6 is a graphical representation of the proposed approach:

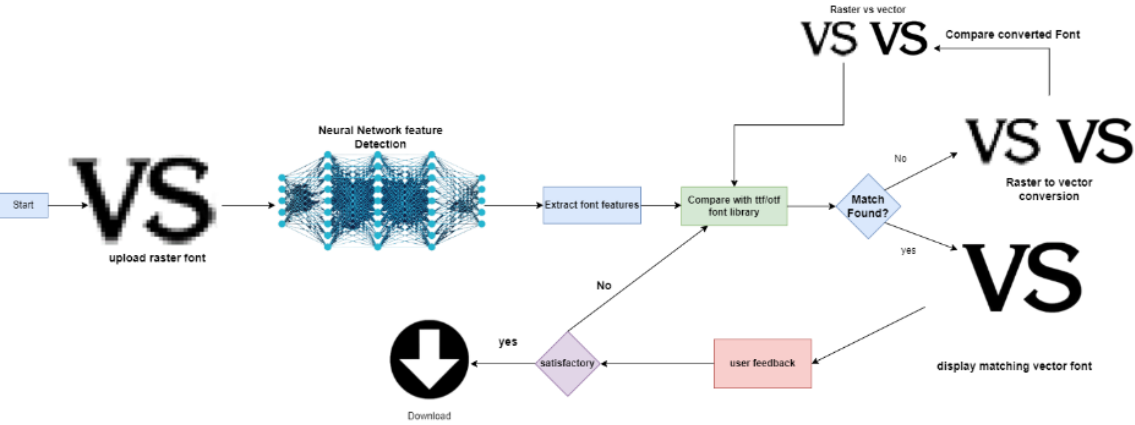


Figure 3.6: Methodology Diagram

Description:

- **Step 1: Feature Extraction:** Input raster and vector fonts are preprocessed to extract relevant visual and structural features.
- **Step 2: Classification:** A supervised machine learning model (CNN) classifies the fonts based on their features.
- **Step 3: Font Matching:** A heuristic algorithm matches the classified raster fonts to the closest vector fonts.
- **Step 4: Website:** A user-friendly website that allows users to submit raster font images and get the similar vector fonts and provide feedback.

Bibliography

- [1] Subhankar Ghosh, Prasun Roy, Saumik Bhattacharya, and Umapada Pal. Large-scale font identification from document images. *Pattern Recognition*, 2020.
- [2] Chen Liu, Jiajun Wu, Pushmeet Kohli, and Yasutaka Furukawa. Raster-to-vector: Revisiting floorplan transformation. *Raster to Vector*, 2018.
- [3] Sinisa Mihajlovic and Ivana Berković. Application and difference of raster and vector graphics. *IX International Conference on Applied Internet and Information Technologies*, 2019.
- [4] Yusuke Nagata, Jinki Otao, and Daichi Haraguchi. Truetype transformer: Character and font style recognition in outline format. *Document Analysis Systems*, 2022.
- [5] Zhangyang Wang and Jianchao Yang. Deepfont: Identify your font from an image. *Image Processing and Computer Vision*, 2015.